

0.0.1. Git [1]

Git es un sistema de control de versiones distribuido que registra los cambios de un proyecto mediante *commits*, permitiendo conservar, comparar y recuperar diferentes versiones de los archivos. A diferencia de un sistema centralizado, cada repositorio Git contiene el historial del proyecto, por lo que el trabajo puede realizarse localmente [1].

Su funcionamiento básico puede describirse mediante tres etapas:

1. **Working Directory:** contiene los archivos sobre los que se está trabajando y donde se realizan las modificaciones.
2. **Staging Area (Index):** contiene los cambios que han sido seleccionados para formar parte del siguiente *commit*.
3. **Local Repository:** almacena permanentemente los cambios registrados mediante *commits*, formando el historial del proyecto.

Las transiciones principales entre estas etapas son:

Estado inicial	Comando	Estado siguiente
Working Directory	git add	Staging Area
Staging Area	git commit	Local Repository
Local Repository	git push	Remote Repository

Por ejemplo, el flujo básico de trabajo es:

```
git add Hola.java
git commit -m "Agrega Hola.java"
git push
```

El comando `git status` permite consultar el estado actual de los archivos y del área de preparación [2].

Fuente [1]: S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>. [Accedido: 21-ago-2026].

0.0.2. Funcionamiento de un programa Java: “Hola mundo” [2]

Un programa Java se escribe inicialmente como código fuente en un archivo `.java`. En este caso, el archivo `Hola.java` contiene la clase `Hola` y su método `main`, que constituye el punto de entrada de la aplicación.

El proceso solicitado puede expresarse textualmente como:

```
Hola.java - javac Hola.java - bytecode - java Hola - Hola mundo
```

Primero, el compilador `javac` transforma el código fuente de `Hola.java` en *bytecode*, almacenado normalmente en `Hola.class`. Este bytecode está diseñado para ser ejecutado por una **Java Virtual Machine (JVM)**, lo que permite mantener la independencia respecto de la arquitectura del sistema [3, 4].

Al ejecutar `java Hola`, el lanzador inicia la JVM, carga la clase `Hola` y llama a su método `main`. Durante la ejecución, la JVM puede interpretar el bytecode y, mediante un compilador **JIT (Just-In-Time)**, convertir código ejecutado frecuentemente en instrucciones nativas optimizadas para el procesador [4].

Para este caso, la ejecución de `System.out.println("Hola mundo");` produce finalmente la salida:

Fuente [2]: Oracle, “The Java[®] Language Specification, Java SE 26 Edition,” Oracle Corporation, 2026. [En línea]. Disponible en: <https://docs.oracle.com/javase/specs/jls/se26/html/>. [Accedido: 21-ago-2026].

Referencias

- [1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>.
- [2] Git Project, “Git Reference.” [En línea]. Disponible en: <https://git-scm.com/docs>.
- [3] Oracle, “The Java[®] Language Specification, Java SE 26 Edition,” Oracle Corporation, 2026. [En línea]. Disponible en: <https://docs.oracle.com/javase/specs/jls/se26/html/>.
- [4] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and A. Smith, *The Java Virtual Machine Specification*, Java SE 26 Edition, Oracle Corporation, 2026. [En línea]. Disponible en: <https://docs.oracle.com/javase/specs/jvms/se26/html/>.